

Inigo Quilez :: articles :: distance functions

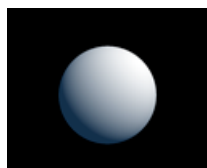


Intro

Here you will find a list of Signed Distance Field implementations for some basic 3D shapes, the formulas to smoothly combine them together needed for building more complex shapes, and some other useful functions, which I used to create all the beautiful images you can find in the [Raymarching Distance Fields](#) article to see what kind of things you can do with SDFs. Please note that this list is not exhaustive, you should read the many [Articles](#) I have under the "SDF & Raymarching" that describe other SDFs shapes and techniques not listed here such as fractal constructions, SDF noise, more rich shape and material blending, lighting, acceleration structures for SDFs and many others. If you are interested in the derivations of these formulas below, you can have a peek in the SDF section in my [video tutorials](#). Lastly, in the code snippets below `dot2(v)` is just short for `dot(v,v)`.

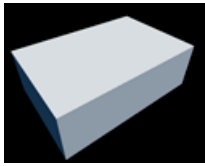
Primitives (<https://www.shadertoy.com/playlist/43cXRI>)

The first block of primitives are all **exact** and **true** SDFs. That is, for each point in space they do measure, precisely, the distance to the closest surface in that shape (with a traditional, Euclidean, L2-norm). Note you'll find many resources in Shadertoy and elsewhere with incorrect SDF implementations that might or might not work to some degree depending on the use case. The ones that follow here, however, are all correct:



Sphere

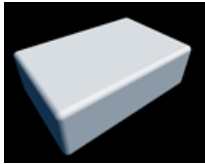
```
float sdSphere( vec3 p, float r )
{
    return length(p) - r;
}
```



Box

<https://www.youtube.com/watch?v=62-pRVZuS5c>

```
float sdBox( vec3 p, vec3 b )
{
    vec3 q = abs(p) - b;
    return length(max(q,0.0)) + min(max(q.x,max(q.y,q.z)),0.0);
}
```



Round Box

```
float sdRoundBox( vec3 p, vec3 b, float r )
{
    vec3 q = abs(p) - b + r;
    return length(max(q,0.0)) + min(max(q.x,max(q.y,q.z)),0.0) - r;
}
```



Box Frame

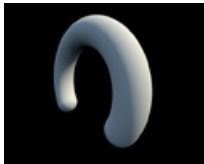
<https://www.shadertoy.com/view/3ljcRh>

```
float sdBoxFrame( vec3 p, vec3 b, float e )
{
    p = abs(p) - b;
    vec3 q = abs(p+e) - e;
    return min(min(
        length(max(vec3(p.x,q.y,q.z),0.0))+min(max(p.x,max(q.y,q.z)),0.0),
        length(max(vec3(q.x,p.y,q.z),0.0))+min(max(q.x,max(p.y,q.z)),0.0)),
        length(max(vec3(q.x,q.y,p.z),0.0))+min(max(q.x,max(q.y,p.z)),0.0));
}
```



Torus

```
float sdTorus( vec3 p, vec2 t )
{
    vec2 q = vec2(length(p.xz)-t.x,p.y);
    return length(q)-t.y;
}
```



Capped Torus

<https://www.shadertoy.com/view/tl23RK>

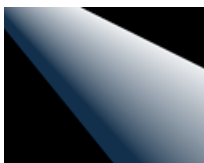
```
float sdCappedTorus( vec3 p, vec2 sc, float ra, float rb)
{
    p.x = abs(p.x);
    float k = (sc.y*p.x>sc.x*p.y) ? dot(p.xy,sc) : length(p.xy);
    return sqrt( dot(p,p) + ra*ra - 2.0*ra*k ) - rb;
}
```



Link

<https://www.shadertoy.com/view/wlXSD7>

```
float sdLink( vec3 p, float le, float r1, float r2 )
{
    vec3 q = vec3( p.x, max(abs(p.y)-le,0.0), p.z );
    return length(vec2(length(q.xy)-r1,q.z)) - r2;
}
```



Infinite Cylinder

```
float sdCylinder( vec3 p, vec3 c )
{
    return length(p.xz-c.xy)-c.z;
}
```



Cone

```
float sdCone( vec3 p, vec2 c, float h )
{
    // c is the sin/cos of the angle, h is height
```

```
// Alternatively pass q instead of (c,h),
// which is the point at the base in 2D
vec2 q = h*vec2(c.x/c.y,-1.0);

vec2 w = vec2( length(p.xz), p.y );
vec2 a = w - q*clamp( dot(w,q)/dot(q,q), 0.0, 1.0 );
vec2 b = w - q*vec2( clamp( w.x/q.x, 0.0, 1.0 ), 1.0 );
float k = sign( q.y );
float d = min(dot( a, a ),dot(b, b));
float s = max( k*(w.x*q.y-w.y*q.x),k*(w.y-q.y) );
return sqrt(d)*sign(s);
}
```



Infinite Cone

```
float sdCone( vec3 p, vec2 c )
{
    // c is the sin/cos of the angle
    vec2 q = vec2( length(p.xz), -p.y );
    float d = length(q-c*max(dot(q,c), 0.0));
    return d * ((q.x*c.y-q.y*c.x<0.0)?-1.0:1.0);
}
```



Plane

```
float sdPlane( vec3 p, vec3 n, float h )
{
    // n must be normalized
    return dot(p,n) + h;
}
```



Hexagonal Prism

```
float sdHexPrism( vec3 p, vec2 h )
{
    const vec3 k = vec3(-0.8660254, 0.5, 0.57735);
    p = abs(p);
```

```

p.xy -= 2.0*min(dot(k.xy, p.xy), 0.0)*k.xy;
vec2 d = vec2(
    length(p.xy-vec2(clamp(p.x,-k.z*h.x,k.z*h.x), h.x))*sign(p.y-h.x),
    p.z-h.y );
return min(max(d.x,d.y),0.0) + length(max(d,0.0));
}

```



Capsule / Line

```

float sdCapsule( vec3 p, vec3 a, vec3 b, float r )
{
    vec3 pa = p - a, ba = b - a;
    float h = clamp( dot(pa,ba)/dot(ba,ba), 0.0, 1.0 );
    return length( pa - ba*h ) - r;
}

```

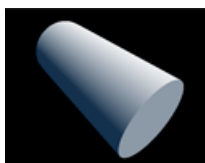


Capsule / Line

```

float sdVerticalCapsule( vec3 p, float h, float r )
{
    p.y -= clamp( p.y, 0.0, h );
    return length( p ) - r;
}

```



Vertical Capped Cylinder

<https://www.shadertoy.com/view/wdXGDr>

```

float sdCappedCylinder( vec3 p, float r, float h )
{
    vec2 d = abs(vec2(length(p.xz),p.y)) - vec2(r,h);
    return min(max(d.x,d.y),0.0) + length(max(d,0.0));
}

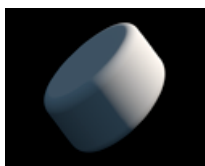
```



Arbitrary Capped Cylinder

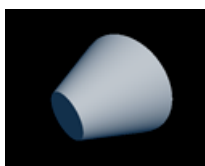
<https://www.shadertoy.com/view/wdXGDr>

```
float sdCappedCylinder( vec3 p, vec3 a, vec3 b, float r )
{
    vec3 ba = b - a;
    vec3 pa = p - a;
    float baba = dot(ba,ba);
    float paba = dot(pa,ba);
    float x = length(pa*baba-ba*paba) - r*baba;
    float y = abs(paba-baba*0.5)-baba*0.5;
    float x2 = x*x;
    float y2 = y*y*baba;
    float d = (max(x,y)<0.0)?-min(x2,y2):(((x>0.0)?x2:0.0)+((y>0.0)?y2:0.0));
    return sign(d)*sqrt(abs(d))/baba;
}
```



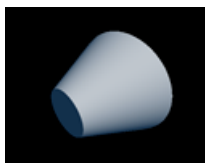
Rounded Cylinder

```
float sdRoundedCylinder( vec3 p, float ra, float rb, float h )
{
    vec2 d = vec2( length(p.xz)-ra+rb, abs(p.y) - h + rb );
    return min(max(d.x,d.y),0.0) + length(max(d,0.0)) - rb;
}
```



Capped Cone

```
float sdCappedCone( vec3 p, float h, float r1, float r2 )
{
    vec2 q = vec2( length(p.xz), p.y );
    vec2 k1 = vec2(r2,h);
    vec2 k2 = vec2(r2-r1,2.0*h);
    vec2 ca = vec2(q.x-min(q.x,(q.y<0.0)?r1:r2), abs(q.y)-h);
    vec2 cb = q - k1 + k2*clamp( dot(k1-q,k2)/dot2(k2), 0.0, 1.0 );
    float s = (cb.x<0.0 && ca.y<0.0) ? -1.0 : 1.0;
    return s*sqrt( min(dot2(ca),dot2(cb)) );
}
```



Capped Cone

<https://www.shadertoy.com/view/tsSXzK>

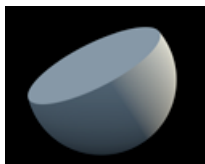
```
float sdCappedCone( vec3 p, vec3 a, vec3 b, float ra, float rb )
{
    float rba = rb-ra;
    float baba = dot(b-a,b-a);
    float papa = dot(p-a,p-a);
    float paba = dot(p-a,b-a)/baba;
    float x = sqrt( papa - paba*paba*baba );
    float cax = max(0.0,x-((paba<0.5)?ra:rb));
    float cay = abs(paba-0.5)-0.5;
    float k = rba*rba + baba;
    float f = clamp( (rba*(x-ra)+paba*baba)/k, 0.0, 1.0 );
    float cbx = x-ra - f*rba;
    float cby = paba - f;
    float s = (cbx<0.0 && cay<0.0) ? -1.0 : 1.0;
    return s*sqrt( min(cax*cax + cay*cay*baba,
                      cbx*cbx + cby*cby*baba) );
}
```



Solid Angle

<https://www.shadertoy.com/view/wtjSDW>

```
float sdSolidAngle( vec3 p, vec2 c, float ra )
{
    // c is the sin/cos of the angle
    vec2 q = vec2( length(p.xz), p.y );
    float l = length(q) - ra;
    float m = length(q - c*clamp(dot(q,c),0.0,ra) );
    return max(l,m*sign(c.y*q.x-c.x*q.y));
}
```



Cut Sphere

<https://www.shadertoy.com/view/stKSzc>

```
float sdCutSphere( vec3 p, float r, float h )
{
    float w = sqrt(r*r-h*h);

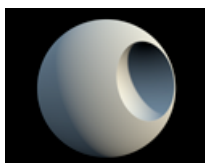
    vec2 q = vec2( length(p.xz), p.y );
    float s = max( (h-r)*q.x*q.x+w*w*(h+r-2.0*q.y), h*q.x-w*q.y );
    return (s<0.0) ? length(q)-r :
        (q.x<w) ? h - q.y :
            length(q-vec2(w,h));
}
```



Cut Hollow Sphere

<https://www.shadertoy.com/view/7tVXRt>

```
float sdCutHollowSphere( vec3 p, float r, float h, float t )
{
    float w = sqrt(r*r-h*h);
    vec2 q = vec2( length(p.xz), p.y );
    return ((h*q.x<w*q.y) ? length(q-vec2(w,h)) :
        abs(length(q)-r) ) - t;
}
```



Death Star

<https://www.shadertoy.com/view/7IVXRt>

```
float sdDeathStar( vec3 p2, float ra, float rb, float d )
{
    float a = (ra*ra - rb*rb + d*d)/(2.0*d);
    float b = sqrt(max(ra*ra-a*a,0.0));

    vec2 p = vec2( p2.x, length(p2.yz) );
    if( p.x*b-p.y*a > d*max(b-p.y,0.0) )
        return length(p-vec2(a,b));
    else
        return max( (length(p)-ra),
            -(length(p-vec2(d,0.0))-rb));
}
```



Round cone

```
float sdRoundCone( vec3 p, float r1, float r2, float h )
{
    float b = (r1-r2)/h;
    float a = sqrt(1.0-b*b);

    vec2 q = vec2( length(p.xz), p.y );
    float k = dot(q,vec2(-b,a));
    if( k<0.0 ) return length(q) - r1;
    if( k>a*h ) return length(q-vec2(0.0,h)) - r2;
    return dot(q, vec2(a,b) ) - r1;
}
```



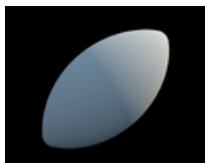
Round Cone

<https://www.shadertoy.com/view/tdXGWr>

```
float sdRoundCone( vec3 p, vec3 a, vec3 b, float r1, float r2 )
{
    vec3 ba = b - a;
    float l2 = dot(ba,ba);
    float rr = r1 - r2;
    float a2 = l2 - rr*rr;
    float il2 = 1.0/l2;

    vec3 pa = p - a;
    float y = dot(pa,ba);
    float z = y - l2;
    float x2 = dot2( pa*l2 - ba*y );
    float y2 = y*y*l2;
    float z2 = z*z*l2;

    // single square root!
    float k = sign(rr)*rr*rr*x2;
    if( sign(z)*a2*z2>k ) return sqrt(x2 + z2) *il2 - r2;
    if( sign(y)*a2*y2<k ) return sqrt(x2 + y2) *il2 - r1;
    return (sqrt(x2*a2*il2)+y*rr)*il2 - r1;
}
```



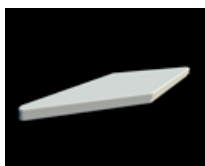
Vesica Segment

<https://www.shadertoy.com/view/Ds2czG>

```
float sdVesicaSegment( in vec3 p, in vec3 a, in vec3 b, in float w )
{
    vec3 c = (a+b)*0.5;
    float l = length(b-a);
    vec3 v = (b-a)/l;
    float y = dot(p-c,v);
    vec2 q = vec2(length(p-c-y*v),abs(y));

    float r = 0.5*l;
    float d = 0.5*(r*r-w*w)/w;
    vec3 h = (r*q.x<d*(q.y-r)) ? vec3(0.0,r,0.0) : vec3(-d,0.0,d+w);

    return length(q-h.xy) - h.z;
}
```



Rhombus

<https://www.shadertoy.com/view/tlVGDC>

```
float sdRhombus( vec3 p, float la, float lb, float h, float ra )
{
    p = abs(p);
    float f = clamp( (la*p.x-lb*p.z+lb*lb)/(la*la+lb*lb), 0.0, 1.0 );
    vec2 w = p.xz - vec2(la,lb)*vec2(f,1.0-f);
    vec2 q = vec2( length(w)*sign(w.x)-ra, p.y-h);
    return min(max(q.x,q.y),0.0) + length(max(q,0.0));
}
```



Octahedron

<https://www.shadertoy.com/view/wsSGDG>

```
float sdOctahedron( vec3 p, float s )
{
    p = abs(p);
    float m = p.x+p.y+p.z-s;
    vec3 q;
```

```

    if( 3.0*p.x < m ) q = p.xyz;
    else if( 3.0*p.y < m ) q = p.yzx;
    else if( 3.0*p.z < m ) q = p.zxy;
    else return m*0.57735027;

    float k = clamp(0.5*(q.z-q.y+s),0.0,s);
    return length(vec3(q.x,q.y-s+k,q.z-k));
}

```



Octahedron - bound (not exact)

```

float sdOctahedron( vec3 p, float s)
{
    p = abs(p);
    return (p.x+p.y+p.z-s)*0.57735027;
}

```



Pyramid

<https://www.shadertoy.com/view/Ws3SDI>

```

float sdPyramid( vec3 p, float h )
{
    float m2 = h*h + 0.25;

    p.xz = abs(p.xz);
    p.xz = (p.z>p.x) ? p.zx : p.xz;
    p.xz -= 0.5;

    vec3 q = vec3( p.z, h*p.y - 0.5*p.x, h*p.x + 0.5*p.y);
    float s = max(-q.x,0.0);
    float t = clamp( (q.y-0.5*p.z)/(m2+0.25), 0.0, 1.0 );
    float a = m2*(q.x+s)*(q.x+s) + q.y*q.y;
    float b = m2*(q.x+0.5*t)*(q.x+0.5*t) + (q.y-m2*t)*(q.y-m2*t);

    float d2 = min(q.y,-q.x*m2-q.y*0.5) > 0.0 ? 0.0 : min(a,b);
    return sqrt( (d2+q.z*q.z)/m2 ) * sign(max(q.z,-p.y));
}

```



Triangle

<https://www.shadertoy.com/view/4sXXRN>

```
float udTriangle( vec3 p, vec3 a, vec3 b, vec3 c )
{
    vec3 ba = b - a; vec3 pa = p - a;
    vec3 cb = c - b; vec3 pb = p - b;
    vec3 ac = a - c; vec3 pc = p - c;
    vec3 nor = cross( ba, ac );

    return sqrt(
        (sign(dot(cross(ba,nor),pa)) +
         sign(dot(cross(cb,nor),pb)) +
         sign(dot(cross(ac,nor),pc))<2.0)
        ?
        min( min(
            dot2(ba*clamp(dot(ba,pa)/dot2(ba),0.0,1.0)-pa),
            dot2(cb*clamp(dot(cb,pb)/dot2(cb),0.0,1.0)-pb) ),
            dot2(ac*clamp(dot(ac,pc)/dot2(ac),0.0,1.0)-pc) )
        :
        dot(nor,pa)*dot(nor,pa)/dot2(nor) );
}
```



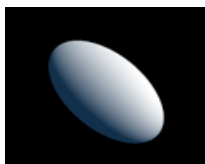
Quad

<https://www.shadertoy.com/view/Md2BWW>

```
float udQuad( vec3 p, vec3 a, vec3 b, vec3 c, vec3 d )
{
    vec3 ba = b - a; vec3 pa = p - a;
    vec3 cb = c - b; vec3 pb = p - b;
    vec3 dc = d - c; vec3 pc = p - c;
    vec3 ad = a - d; vec3 pd = p - d;
    vec3 nor = cross( ba, ad );

    return sqrt(
        (sign(dot(cross(ba,nor),pa)) +
         sign(dot(cross(cb,nor),pb)) +
         sign(dot(cross(dc,nor),pc)) +
         sign(dot(cross(ad,nor),pd))<3.0)
        ?
        min( min( min(
            dot2(ba*clamp(dot(ba,pa)/dot2(ba),0.0,1.0)-pa),
            dot2(cb*clamp(dot(cb,pb)/dot2(cb),0.0,1.0)-pb) ),
            dot2(dc*clamp(dot(dc,pc)/dot2(dc),0.0,1.0)-pc) ),
            dot2(ad*clamp(dot(ad,pd)/dot2(ad),0.0,1.0)-pd) )
        :
        dot(nor,pa)*dot(nor,pa)/dot2(nor) );
}
```

The following block of functions are **NOT correct and true SDFs**. Instead they do approximate SDFs in some sense or another, so sometimes they can be used with certain rendering/collision algorithms. In particular, they are all lower bounds and all produce zero exactly at the surface of the shape, so they can be used with basic raymarching.



Ellipsoid - lower bound

<https://www.shadertoy.com/view/tdS3DG>

```
float sdEllipsoid( vec3 p, vec3 r )
{
    float k0 = length(p/r);
    float k1 = length(p/(r*r));
    return k0*(k0-1.0)/k1;
}
```



Triangular Prism - lower bound

```
float sdTriPrism( vec3 p, vec2 h )
{
    vec3 q = abs(p);
    return max(q.z-h.y, max(q.x*0.866025+p.y*0.5, -p.y)-h.x*0.5);
}
```

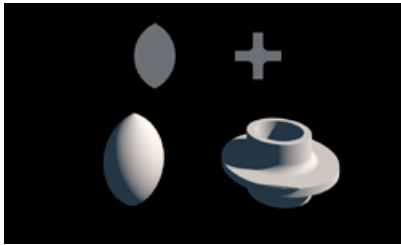
Creating more 3D SDFs, from 2D SDFs

The list above is definitely not exhaustive, many other shapes are easy to describe through a simple SDF. One simple way to do so, is to take any **2D SDF** and either revolve it or extrude it. This is really easy to do, and has the advantage that if the the 2D SDF we start with is an exact SDF, the resulting 3D volume is exact as well.

This is interesting for a couple of reasons: first, creating a shape through 3D boolean operations of basic forms does **not** produce an exact SDF (we'll talk about this later in this article), while doing it as a revolution or extrusion of a 2D shape does produce the correct SDF. Secondly, doing 3D boolean operations generates suboptimal code since it does not reuse common expressions across primitives. So doing revolution or extrusion of a 2D shapes produces the correct SDF and is also faster to compute.

You can find code to do extrusion and revolution below, and also here:

<https://www.shadertoy.com/view/4lyfzw>



```
float opRevolution( in vec3 p, in sdf2d primitive, in float o )
{
    vec2 q = vec2( length(p.xz) - o, p.y );
    return primitive(q)
}
```



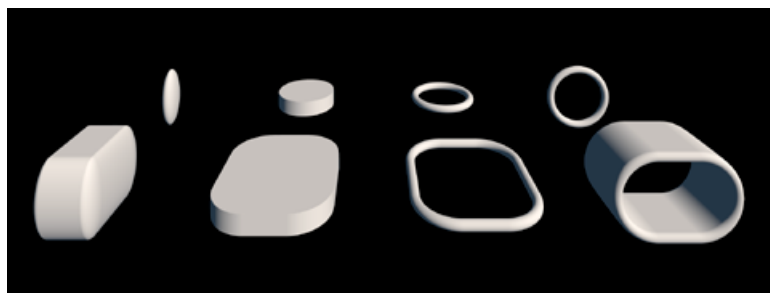
```
float opExtrusion( in vec3 p, in sdf2d primitive, in float h )
{
    float d = primitive(p.xy)
    vec2 w = vec2( d, abs(p.z) - h );
    return min(max(w.x,w.y),0.0) + length(max(w,0.0));
}
```

Creating more 3D SDFs, from 3D SDFs

It is also possible to create new types of 3D primitives from other 3D primitives. Here are some examples:

Elongation - bound/exact

Elongating is a useful way to construct new shapes. It basically splits a primitive in two (four or eight), moves the pieces apart and connects them. It is a perfect distance preserving operation, it does not introduce any artifacts in the SDF. Some of the basic primitives above use this technique. For example, the Capsule is an elongated Sphere along an axis really. You can find code here: <https://www.shadertoy.com/view/Ml3fWj>



```
float opElongate( in sdf3d primitive, in vec3 p, in vec3 h )
{
    vec3 q = p - clamp( p, -h, h );
    return primitive( q );
}

float opElongate( in sdf3d primitive, in vec3 p, in vec3 h )
{
    vec3 q = abs(p)-h;
    return primitive( max(q,0.0) ) + min(max(q.x,max(q.y,q.z)),0.0);
}
```

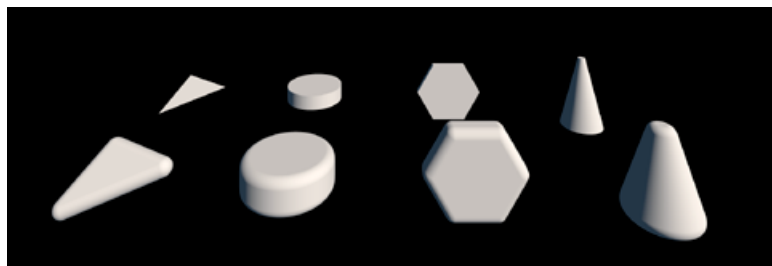
The reason I provide implementations is the following. For 1D elongations, the first function works perfectly and gives exact exterior and interior distances. However, the first implementation produces a small core of zero distances inside the volume for 2D and 3D elongations. Depending on your application that might be a problem. One way to create exact interior distances all the way to the very elongated core of the volume, is the following, which is in languages like GLSL that don't have function pointers or lambdas need to be implemented a bit differently (check the code linked about in Shadertoy to see one example).

Rounding/Inflating - exact

Rounding a shape is as simple as subtracting some distance (jumping to a different isosurface). The rounded box above is an example, but you can apply it to cones, hexagons or any other shape like the cone in the

image below. If you happen to be interested in preserving the overall volume of the shape, most of the time it's pretty easy to shrink the source primitive by the same amount we are rounding it by. You can find code here:

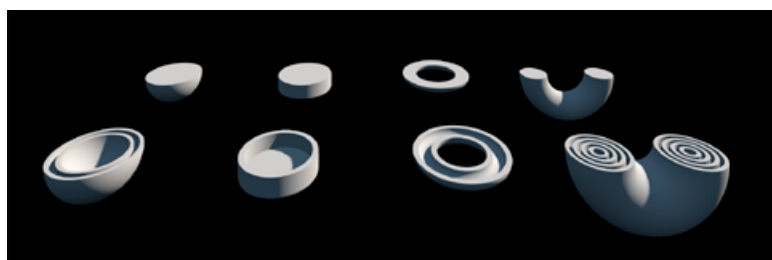
<https://www.shadertoy.com/view/Mt3BDj>



```
float opRound( in sdf3d primitive, in float rad )
{
    return primitive(p) - rad
}
```

Onion - exact

For carving interiors or giving thickness to primitives, without performing expensive boolean operations (see below) and without distorting the distance field into a bound, one can use "onion-ing". You can use it multiple times to create concentric layers in your SDF. You can find code here: <https://www.shadertoy.com/view/MlcBDj>

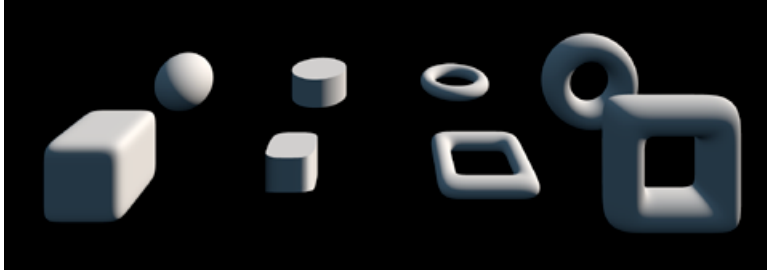


```
float opOnion( in float sdf, in float thickness )
{
    return abs(sdf) - thickness;
}
```

Change of Metric - bound

Most of these functions can be modified to use other norms than the Euclidean. By replacing `length(p)`, which

computes $(x^2+y^2+z^2)^{1/2}$ by $(x^n+y^n+z^n)^{1/n}$ one can get variations of the basic primitives that have rounded edges rather than sharp ones. However **I do not recommend** using this technique other than for toy demos, since these are not true SDFs anymore but lower bounds, meaning that they require more raymarching steps than usual. Since they only give a bound to the real SDF, this kind of primitive alteration also doesn't play well with shadows and occlusion algorithms that rely on true SDFs for measuring distance to occludes. You can find the code here: <https://www.shadertoy.com/view/ltofDj>



```
float length2( vec3 p )
{
    p=p*p; return
    sqrt( p.x+p.y+p.z );
}

float length6( vec3 p )
{
    p=p*p*p; p=p*p; return
    pow(p.x+p.y+p.z,1.0/6.0);
}

float length8( vec3 p )
{
    p=p*p; p=p*p; p=p*p; return
    pow(p.x+p.y+p.z,1.0/8.0);
}
```

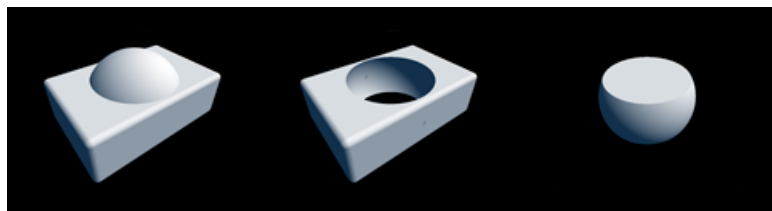
Primitive combinations

Sometimes you cannot simply elongate, round or onion a primitive, and you need to combine, carve or intersect basic primitives. Given the SDFs **a** and **b** of two primitives, you can use the following operators to combine together.

Union, Subtraction, Intersection - exact/bound, bound, bound

These are the most basic combinations of pairs of primitives you can do. They correspond to the basic boolean operations. **Please note** that the Xor and the Union of two SDFs produces a true SDF, but not the Subtraction or Intersection. To make it more subtle, this is only true in the exterior of the SDF (where distances are positive) and not in the interior. You can learn more in [this article about Xor](#), this and how to work around the incorrect interior distances of the Union in the article about ["Interior Distances"](#). Also note that

opSubtraction() is not commutative and depending on the order of the operand it will produce different results.

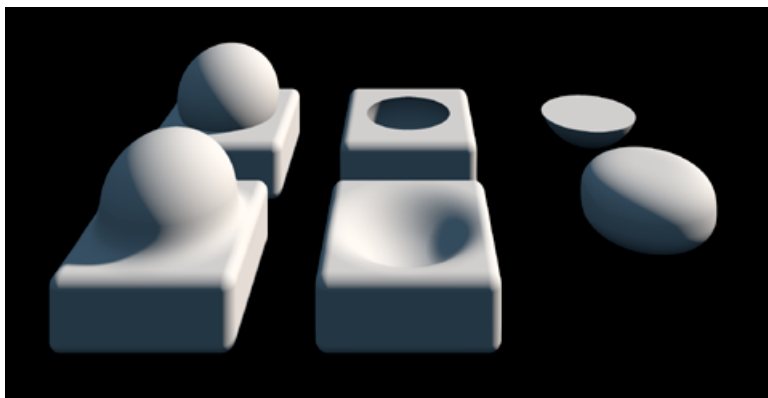


```
float opUnion( float a, float b )
{
    return min(a,b);
}
float opSubtraction( float a, float b )
{
    return max(-a,b);
}
float opIntersection( float a, float b )
{
    return max(a,b);
}
float opXor( float a, float b )
{
    return max(min(a,b),-max(a,b));
}
```

Smooth Union, Subtraction and Intersection - bound, bound, bound

Blending primitives is a really powerful tool - it allows to construct complex and organic shapes without the geometrical seams that normal boolean operations produce. There are many flavors of such operations, but the basic ones try to replace the min() and max() functions used in the opUnion, opSubtraction and opIntersection above with smooth versions. They all accept an extra parameter called **k** that defines the size of the smooth transition between the two primitives. It is given in actual distance units. You can find more details in the [smooth minimum article](https://www.shadertoy.com/view/lt3BW2) in this same site. You can code here:

<https://www.shadertoy.com/view/lt3BW2>



```
float opSmoothUnion( float a, float b, float k )
{
    k *= 4.0;
    float h = max(k-abs(a-b),0.0);
    return min(a, b) - h*h*0.25/k;
}

float opSmoothSubtraction( float a, float b, float k )
{
    return -opSmoothUnion(a,-b,k);

    // k *= 4.0;
    // float h = max(k-abs(-a-b),0.0);
    // return max(-a, b) + h*h*0.25/k;
}

float opSmoothIntersection( float a, float b, float k )
{
    return -opSmoothUnion(-a,-b,k);

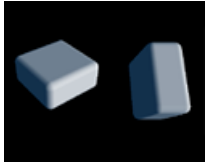
    // k *= 4.0;
    // float h = max(k-abs(a-b),0.0);
    // return max(a, b) + h*h*0.25/k;
}
```

Positioning

Placing primitives in different locations and orientations in space is a fundamental operation in designing SDFs. While rotations, uniform scaling and translations are exact operations, non-uniform scaling distorts the Euclidean spaces and can only be bound. Therefore I do not include it here.

Rotation/Translation - exact

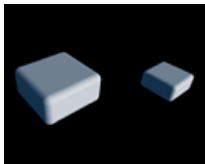
Since rotations and translation don't compress nor dilate space, all we need to do is simply to transform the point being sampled with the inverse of the transformation used to place an object in the scene. This code below assumes that transform encodes only a rotation and a translation (as a 3x4 matrix for example, or as a quaternion and a vector), and that it does not contain any scaling factors in it.



```
vec3 opTx( in vec3 p, in transform t, in sdf3d primitive )
{
    return primitive( invert(t)*p );
}
```

Scale - exact

Scaling an object is slightly more tricky since that compresses/dilates spaces, so we have to take that into account on the resulting distance estimation. Still, it's not difficult to perform, although it only works with uniform scaling. Non uniform scaling is not possible (while still getting a correct SDF):



```
float opScale( in vec3 p, in float s, in sdf3d primitive )
{
    return primitive(p/s)*s;
}
```

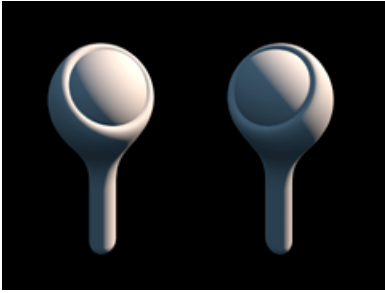
Symmetry and repetition

Something beautiful with procedural SDFs is that creating multiple copies of the same object can be done easily at no memory or performance cost. By making the SDF function itself symmetric or periodic we get automatic instancing in constant time, with just a few lines of code.

Symmetry - bound and exact

Symmetry is useful, since many things around us are symmetric, from humans, animals, vehicles, instruments, furniture, ... Oftentimes, one can take shortcuts and only model half or a quarter of the desired shape, and get it duplicated automatically by using the absolute value of the domain coordinates before

evaluation. For example, in the image below, there's a single object evaluation instead of two. You have to be aware however that the resulting SDF might not be an exact SDF but a bound, if the object you are mirroring crosses the mirroring plane.



```
float opSymX( in vec3 p, in sdf3d primitive )
{
    p.x = abs(p.x);
    return primitive(p);
}

float opSymXZ( in vec3 p, in sdf3d primitive )
{
    p.xz = abs(p.xz);
    return primitive(p);
}
```

Infinite and limited Repetition

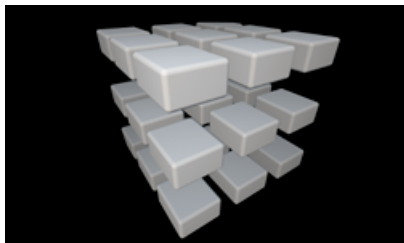
Domain repetition is a very useful operator, since it allows you to create infinitely many primitives with a single object evaluation:



```
float opRepetition( in vec3 p, in vec3 s, in sdf3d primitive )
{
    vec3 q = p - s*round(p/s);
    return primitive( q );
}
```

In this code **s** is the spacing between the instances. This function above will only work for symmetric shapes (with respect to the repeating tile boundaries), generally raymarchers will not render objects properly if this function is used naively as is. To learn how to make it work for arbitrary SDFs without artifacts, and to learn more about all sort of repetition patterns, like circular, rectangular, please see the [article on Domain Repetition](https://iquilezles.org/articles/distfunctions/).

Limited domain repetition is similar to the above but it limits the number of instances, which is useful for human made things where we usually don't have infinite of anything. If you want to use limited repetition with non symmetric shapes, please read the [article on Domain Repetition](#) to learn how to fix them.



```
vec3 opLimitedRepetition( in vec3 p, in float s, in vec3 l, in sdf3d primitive )
{
    vec3 q = p - s*clamp(round(p/s),-1,1);
    return primitive( q );
}
```

The article linked above also explains how to use instance identifiers to perform shape and color variations such that each shape in the repetition grid looks different, among other things.

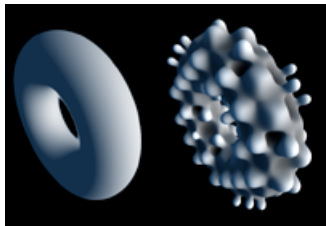
Deformations and distortions

Deformations and distortions allow to enhance the shape of primitives or even fuse different primitives together. The operations usually distort the distance field and make it non-Euclidean anymore, so one must be careful when raymarching them, you will probably need to decrease your step size, if you are using a raymarcher to sample this. In principle one can compute the factor by which the step size needs to be reduced (inversely proportional to the compression of the space, which is given by the Jacobian of the deformation function). But even with dual numbers or automatic differentiation, it's usually just easier to find the constant by hand for a given primitive.

I'd say that while it is tempting to use a distortion or displacement to achieve a given shape, and I often use them myself of course, it is sometimes better to get as close to the desired shape with actual exact Euclidean primitive operations (elongation, rounding, onion-ing, union) or tight bounded functions (intersection, subtraction) and then only apply as small of a distortion or displacement as possible. That way the field stays as close as possible to an actual distance field, and the raymarcher will be faster.

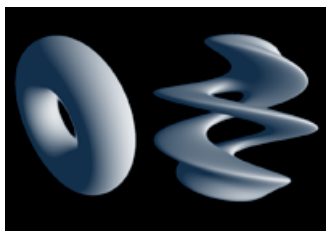
Displacement

The displacement example below is using $\sin(20 \cdot p.x) \cdot \sin(20 \cdot p.y) \cdot \sin(20 \cdot p.z)$ as displacement pattern, but you can of course use anything you might imagine.



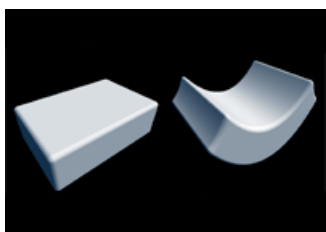
```
float opDisplace( in sdf3d primitive, in vec3 p )
{
    float d1 = primitive(p);
    float d2 = displacement(p);
    return d1+d2;
}
```

Twist



```
float opTwist( in sdf3d primitive, in vec3 p )
{
    const float k = 10.0; // or some other amount
    float c = cos(k*p.y);
    float s = sin(k*p.y);
    mat2 m = mat2(c,-s,s,c);
    vec3 q = vec3(m*p.xz,p.y);
    return primitive(q);
}
```

Bend



```
float opCheapBend( in sdf3d primitive, in vec3 p )
{
    const float k = 10.0; // or some other amount
    float c = cos(k*p.x);
    float s = sin(k*p.x);
    mat2 m = mat2(c,-s,s,c);
    vec3 q = vec3(m*p.xy,p.z);
    return primitive(q);
}
```

<https://www.shadertoy.com/view/Xds3zN>:

